

partial failure - some parts of the system are broken even though other parts of the system are working fine

(can lead to **non-deterministic behavior**)

Unreliable Networks :

shared-nothing systems - a system composed of individual machines that can only communicate via network

The internet and most internal networks in datacenters (often ethernet) are **asynchronous packet networks**. In this kind of network, one node can send a message (a packet) to another node, but the network gives no guarantees if/when the packet will arrive.

there are many things that can go wrong
↓

If you send a request and do not get a response, it's impossible to distinguish whether :

- a) the request was lost,
- b) the remote node is down, or
- c) the response was lost

Detecting Faults

Many systems need to automatically detect faulty nodes (think load balancer or if the leader fails in a single-leader replication configured database)

Uncertainty about networks makes it difficult to tell whether a node is working or not; the only way to know that a request was successful is by receiving a positive response from the application itself

So, you have to assume you will get no response, therefore, you can retry a few times, wait for a timeout to elapse, and eventually declare the node dead

Timeouts and Unbounded Delays

If timeout only way to detect fault, how long do we make timeout?

- too short can result in prematurely declaring a node dead

If your timeout is low, it only takes a transient spike in round-trip times to throw the system off-balance; it's not enough for your system to be fast most of the time

Network Congestion and Queueing:

The variability of packet delays is most often due to queueing:

- if several nodes simultaneously send packets to the same destination, the network switch will queue them up; if the switch fills up, the packet is dropped, so it needs to be resent
- When packet reaches machine, if all CPU cores are busy then the OS will queue the incoming request
- TCP performs **congestion avoidance** in which a node limits its own rate of sending to avoid overloading a network link, resulting in additional queueing at the sender

Async vs Sync Networks:

When you make a call over a traditional fixed-line telephone network, it establishes a **circuit**; a fixed amt of reserved bandwidth which nobody else can use while the circuit is established



this kind of network is **synchronous**, does not suffer from queueing ∴ the max end-to-end latency is fixed

↳ bounded delay

TCP is asynchronous (**packet switched protocol**) and do not use circuits

Datacenter networks and the internet use packet switched networks because they are optimized for **bursty traffic**



requesting web page, sending an email,
transferring a file, etc



does not have any bandwidth req. just want to complete ASAP

If you wanted to do those above mentioned tasks with a circuit, you'd have to guess the amount of bandwidth allocation:

- guess too low: your transfer will be very slow
- guess too high: the circuit cannot be set up bc

the network cannot allow a circuit to
be created if the bandwidth allocation
cannot be guaranteed.

Unreliable Clocks

Possible to synchronize clocks to some degree, the most commonly used method is **Network Time Protocol (NTP)**, where the computer clock is adjusted to the time reported by a group of servers.

Time-of-day Clocks - returns the current date and time according to some calendar

- not suitable for measuring elapsed time
- can jump forward/backward in time if it is too far ahead/behind the NTP server

Monotonic Clocks - suitable for measuring elapsed time

- guaranteed to always move forward
- in a distributed system, using this clock to measure elapsed time is usually fine

Relying on Synchronized Clocks:

Incorrectly configured clocks easily go unnoticed, and the result is likely to be silent and subtle data loss, not a system crash

- ∴ need to use software that monitors the clock offset between all machines

Timestamps for ordering events:

- LWW uses timestamps for conflict resolution

Problems:

- a node with a lagging clock will never be able to overwrite values previously written by a node with a fast clock until the clock skew between the nodes has elapsed

↳ LWW always picks last write, this scenario can cause arbitrary amt of data loss without any error

- cannot distinguish between writes that occurred sequentially in quick succession and truly concurrent writes

logical clocks are a safer alternative for ordering events

The Truth is Defined by the Majority

Many distributed systems rely on a **quorum**, that is, voting among the nodes $\rightarrow$ decisions require some minimum number of votes from several nodes (most commonly an absolute majority of more than half the nodes)

The leader and the lock

Frequently, a system requires there to be only one of something:

- one node is a leader for a partition
- only one client can hold a lock for a resource
- only one user can have a particular username

Implementing this in a distributed system requires care: even if a node believes it is the "chosen one" (leader of partition, holder of the lock, etc) that doesn't mean a quorum of nodes agree.

Byzantine Faults

We assume that nodes are unreliable but honest: if a node does respond, it is telling the truth

If a node is lying (sending arbitrary faulty or corrupted messages) that behavior is known as **Byzantine fault** and the problem of reaching consensus in such an environment is known as **Byzantine Generals Problem**

A system is **Byzantine fault-tolerant** if it continues to operate correctly even if some nodes are malfunctioning or malicious attackers are interfering with the network

System Model and Reality

System model - abstraction that describes things an algorithm may assume

Three system models related to timing assumptions:

Synchronous Model:

- network delay, pauses, and clock drift will never exceed some fixed upper bound
- not realistic

Partially Synchronous Model

- system behaves like a synchronous system most of the time; sometimes exceeds the bounds for network delay, process pause, and clock drift
- realistic model

Asynchronous Model

- in this model, an algorithm is not allowed to make any assumption about timing

Other models related to node failures also exist

Most useful model to mimic real systems is the **partially synchronous model with crash-recovery faults**

Useful because they distill down the complexity of real systems to a manageable set of faults that we can reason about. Then prove algorithms correct by showing that their properties hold in some system model.

